

Biểu thức và nhập xuất

Nguyễn Huyền Châu – AI Lab/TLU

Nội dung

- Các vấn đề về định kiểu
- Định dạng nhập xuất
- Làm việc với xâu và ký tự
- Một số hàm hữu ích
- Trò chơi đoán số (tiếp)

Các vấn đề về định kiểu

- ❑ An toàn kiểu, vi phạm kiểu
- ❑ Ép kiểu ngầm định, ép kiểu tường minh
- ❑ Tràn, hụt

An toàn kiểu

Kiểu quyết định ý nghĩa và khả năng tồn tại của:

- Dữ liệu:
 - hằng, biến phải khai báo kiểu, literal cũng có kiểu
 - dữ liệu được hiểu thế nào tùy kiểu đã xác định
 - dữ liệu nằm trong phạm vi nào cũng do kiểu quyết định
- Thao tác:
 - Toán hạng phải hợp kiểu mới kết hợp được toán tử
 - Cả khi kết hợp được, ra kết quả thế nào cũng tùy kiểu toán hạng
- Tập hợp các ràng buộc liên quan đến kiểu làm thành sự định kiểu hay an toàn kiểu (typesafety) của ngôn ngữ

Ví dụ 1a: Kiểu quyết định ý nghĩa

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      char c;
7      int i;
8      double d;
9      cout << " i: " << i << " c: " << c
10     << " d: " << d << '\n';
11
12     char c1 = 'A';
13     cout << c1 << '\n'; // A
14     int i1 = c1;
15     cout << i1 << '\n'; // 65
16
17     unsigned int unsignedInt = -1;
18     int signedInt = -1;
19     cout << "Unsigned -1 = " << unsignedInt << '\n'
20         << "Signed -1 = " << signedInt << '\n';
21
22     return 0;
23 }
```

Biến không khởi gán có thể có giá trị khó kiểm soát

i: 32766 c: d: 0
A
65
Unsigned -1 = 4294967295
Signed -1 = -1

Trong bộ nhớ, 'A' và 65 cùng lưu như một chuỗi bit nhị phân. Chỉ có kiểu của biến mới cho biết ý nghĩa cụ thể của chuỗi bit đó

Tương tự, -1 mang ý nghĩa khác nhau tùy theo được khai báo là số có dấu hay không dấu

Vi phạm kiểu

- C++ cho phép vi phạm kiểu nhất định
 - Mã lệnh dễ viết, linh hoạt hơn
 - Khi gặp sự không tương thích, C++ sẽ cố sửa chữa bằng cách ép kiểu tự động (ép kiểu ngầm định), nếu không được nó sẽ báo lỗi để ta tự sửa (ép kiểu tường minh)
 - Khi gặp sự ngoài phạm vi, C++ cũng cố ép giá trị về trong phạm vi (tình huống tràn, hụt).
- Có lúc C sửa và như ta muốn, có lúc sửa mà không như ta muốn, có lúc không sửa được và báo lỗi để ta tự sửa.

Ép kiểu ngầm định

- Thực hiện bởi trình biên dịch *khi có thể*
- Toán tử số học: kiểu thấp được ép sang kiểu cao.
- Toán tử gán: vế phải được ép sang kiểu vế trái. Việc ép thực hiện *sau khi* vế phải đã tính xong.
 - ❑ `float a = int1/int2` có ép kiểu nhờ phép gán, song vẫn không làm vế phải chia kiểu float, vì ép kiểu xảy ra sau phép chia.

```
int a =5;
float b=1.5;
int sum = a + b ;
cout << a << " + " << b
    << " = " << a + b << "\n";
cout << "int sum = " << a << " + " << b
    << " = " << sum << "\n";
```

```
5 + 1.5 = 6.5
int sum = 5 + 1.5 = 6
```

Ép kiểu tường minh

- Thực hiện bởi người lập trình theo cú pháp:
 <kiểu mới> (giá trị cần ép kiểu)

```
float number = 3.5;  
intVal = int(number); //Val=3, number=3.5
```

- VD: Các biến dưới đây được gán giá trị bao nhiêu?

```
float perMonth1 = float(books)/months  
float perMonth2 = float(books/months)  
float perMonth3 = books/float(months)  
//int books = 15, months = 6;
```

Ép kiểu có thay đổi biến?

Không, nó thực hiện trên giá trị tức thời của biến, không phải trên biến (giống chỉnh sửa ảnh chụp đồ vật, không phải chỉnh đồ vật)

Ví dụ 1b: Ép kiểu

```
1  #include <iostream>
2  #include <typeinfo>
3  #include <iomanip>
4  using namespace std;
5  int main()
6  {
7      cout << setprecision(2) << setiosflags(ios::fixed | ios::showpoint);
8      const float PI = 3.14, E = 2.71;
9      int int1 = 3, int2 = 2, int3, intPI;
10     float float1 = 9, float2=7.51;
11
12     // IMPLICIT TYPECASTING
13     // value and type of a real literal divided by an integer
14     cout << " real literal and int: " << 5.0 << " / " << int1 << " = " << 5.0 / int1
15         << " and has type " << typeid(5.0/int1).name() << "\n";
16
17     // value and type of a float constant multiplied by an integer
18     cout << " float const and int : " << PI << " * " << int1 << " = " << PI * int1
19         << " and has type " << typeid(PI*int1).name() << "\n";
20
21     // value and type of a float variable subtracted by an integer
22     cout << " float var and int : " << float1 << " - " << int1 << " = " << float1-int1
23         << " and has type " << typeid(float1-int1).name() << "\n";
```

<typeinfo> chứa hàm check kiểu còn <iomanip> có hàm hiển thị số thực dạng phẩy thập

Hiển thị số thực dạng dấu phẩy thập, luôn có dấu

Trong phép tính số học 2 ngôi, toán hạng có kiểu thấp hơn bị ngầm định đổi về kiểu cao: ở đây các toán hạng int đều bị chuyển về kiểu của toán hạng float

```

24
25 // value and type of a float addition divided by an integer
26 cout << " float expression and int : "
27     << "(" << 6.0 << " + " << int2 << ")" << " / " << int2 << " = " << (6.0 + int2)/int2
28     << " and has type " << typeid((6.0+int2)/int2).name() << "\n";
29
30 // value and type of an float expression assigned to an integer
31 int3 = (6.0+int2)/int2;
32 cout << " assignment int3 = " << "(" << 6.0 << " + " << int2 << ")" << " / " << int2
33     << " = " << int3 << " and has type " << typeid(int3).name() << "\n";
34 //the right operator's value remains but its type
35 //has been casted to integer
36

```

Toán hạng int sẽ chuyển về kiểu của toán hạng float

Còn trong phép gán, thì kiểu của giá trị về phải luôn đổi về kiểu biến về trái: nên ở đây về phải float chuyển về kiểu int

```

real literal and int: 5.00 / 3 = 1.67 and has type d
float const and int : 3.14f * 3 = 9.42 and has type f
float var and int : 9.00f - 3 = 6.00 and has type f
float expression and int : (6.00 + 2) / 2 = 4.00 and has type d
assignment int3 = (6.00 + 2) / 2 = 4 and has type i

```

```
36
37 // IMPOSSIBLE IMPLICIT TYPECASTING
```

```
38 cout << " int3 % int2 : " << int3 << " % " << int2 << " = " << int3%int2
```

```
39 |   << " and has type " << typeid(int3%int2).name() << "\n";
```

```
40 //valid
```

```
41 int3 % int2 : 4 % 2 = 0 and has type i
```

```
42 cout << "float % int: " << float2 << "f % " << int2
```

```
43 |   << " = " << float2%int2 << " and has type "
```

```
44 |   << typeid(float2%int2).name() << "\n";
```

```
45 //error
```

```
46
47 cout << " float expression % int2 : "
```

```
48 |   << "(" << float1 << "f - " << int1 << ")" << " % " << int2
```

```
49 |   << " = " << (float1-int1)%int2 << " and has type "
```

```
50 |   << typeid((float1-int1)%int2).name() << "\n";
```

```
51 //error
52
```

Các toán hạng đều kiểu
int và phép % hợp lệ.

Các toán hạng đều sẽ kiểu
float, phép % không hợp lệ.

```
typecasting.cpp:43:27: error: invalid operands to binary expression ('float' and 'int')
    << " = " << float2%int2 << " and has type "
```

```
typecasting.cpp:44:25: error: invalid operands to binary expression ('float' and 'int')
    << typeid(float2%int2).name() << "\n";
```

```
typecasting.cpp:49:34: error: invalid operands to binary expression ('float' and 'int')
    << " = " << (float1-int1)%int2 << " and has type "
```

```
typecasting.cpp:50:32: error: invalid operands to binary expression ('float' and 'int')
    << typeid((float1-int1)%int2).name() << "\n";
```

```
4 errors generated.
```

```

36
37 // IMPOSSIBLE IMPLICIT TYPECASTING
38 cout << " int3 % int2 : " << int3 << " % " << int2 << " = " << int3%int2
39 |   << " and has type " << typeid(int3%int2).name() << "\n";
40 //valid
41
42 // cout << "float % int: " << float2 << "f % " << int2
43 //   << " = " << float2%int2 << " and has type "
44 //   << typeid(float2%int2).name() << "\n";
45 // //error
46
47 // cout << " float expression % int2 : "
48 //   << "(" << float1 << "f - " << int1 << ")" << " % " << int2
49 //   << " = " << (float1-int1)%int2 << " and has type "
50 //   << typeid((float1-int1)%int2).name() << "\n";
51 // //error
52
53 // EXPLICIT TYPECASTING
54 cout << " int(float) % int : " << "int(" << float2 << "f) % " << int2 << " = "
55 |   << int(float2) %int2 << " and has type " << typeid(int (float2)%int2).name() << "\n";
56 //no error
57
58 cout << " int(float expression) % int2 : " << "int(" << float1 << "f - " << int1 << ")"
59 |   << " % " << int2 << " = " << int(float1-int1)%int2
60 |   << " and has type " << typeid(int(float1-int1)%int2).name() << "\n";
61 // no error
62

```

Ép kiểu tương minh

```

int3 % int2 : 4 % 2 = 0 and has type i
int(float) % int : int(7.51f) % 2 = 1 and has type i
int(float expression) % int2 : int(9.00f - 3) % 2 = 0 and has type i

```



```

61 // no error
62
63 // TYPECASTING EFFECT
64 // typecasting doesnt change the object,
65 // just return a modified version of the object
66
67 // implicitly typecast PI to integer
68 intPI = PI;
69 cout << " intPI = PI = " << intPI << "\n";
70 // explicitly typecast E to integer
71 cout << " int (E) = " << int(E) << "\n";
72
73 cout << " After typecasting : \n"; //they re the same
74 cout <<" float2 still is: " << float2 << "\n";
75 cout <<" PI still is: " << PI << "\n";
76 cout <<" E still is: " << E << "\n";
77
78 return 0;
79 }

```

Hoàn toàn hợp lệ khi ép kiểu hằng, bởi ép kiểu không hề thay đổi đối tượng, chỉ tạo ra một bản sao của đối tượng có kiểu khác

```

intPI = PI = 3
int (E) = 2
After typecasting :
float2 still is: 7.51
PI still is: 3.14
E still is: 2.71

```

Ví dụ 1c: Kiểu quyết định phạm vi

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int a = 20000;
7      char c = a;
8      int b = c;
9      if (a != b) // != means "not equal"
10         cout << "oops!: " << a << "!=" << b << '\n';
11     else
12         cout << "Wow! We have large characters\n";
13
14     return 0;
15 }
```

Giá trị 2000 của a khi gán cho kiểu char bị chuyển kiểu và trở thành 32; khi gán cho b kiểu int thì vẫn giữ kiểu int và giá trị 32. Vậy b = c = 32. Còn sau mọi phép gán cho b và c, a luôn giữ giá trị 20000. Nên a không bằng b.

oops!: 20000!=32

Phạm vi của kiểu (nhắc lại)

Kiểu	Mô tả	Kích thước thông dụng (byte)	Phạm vi (tương ứng với kích thước)
char	ký tự / số nguyên nhỏ	1	các kí tự ASCII signed char: -128 → 127, hoặc unsigned char: 0 → 255
bool	giá trị Boolean	1	true hoặc false
short	số nguyên	2	signed short: -32767 → 32767 unsigned short: 0 → 65536
int	số nguyên lớn	4	signed int: 2147483648 → 2147483647 unsigned int: 0 → 4294967296
long	số nguyên rất lớn	4	signed long: 2147483648 → 2147483647 unsigned long: 0 → 4294967296
float	số thực	4	+/- 1.4023x10 ⁻⁴⁵ → 3.4028x10 ⁺³⁸
double	số thực với độ chính xác cao	8	+/- 4.9406x10 ⁻³²⁴ → 1.7977x10 ³⁰⁸
long double	số thực với độ chính xác rất cao	8	+/- 4.9406x10 ⁻³²⁴ → 1.7977x10 ³⁰⁸

Tràn và hụt

- Xảy ra khi một biến được gán giá trị quá lớn hay quá nhỏ so với phạm vi kiểu của nó
- Tràn (overflow) – Khi biến được gán giá trị lớn vượt quá phạm vi kiểu
- Hụt (underflow) – Khi biến được gán giá trị nhỏ quá phạm vi kiểu

Ví dụ 1d: Tràn và hụt

```
1  // This program demonstrates underflow and overflow
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      // Integer underflow and overflow
8      short testVar = 32767;
9      cout << testVar << endl;
10     testVar = testVar + 1; // overflow testVar
11     cout << testVar << endl;
12     testVar = testVar - 1; // underflow testVar
13     cout << testVar << endl << endl;
14
15     //Floating point overflow and underflow:
16     float test;
17     test = 2.0e38 * 1000; // overflow test
18     cout << test << endl;
19     test = 2.0e-38 / 2.0e38; // underflow test
20     cout << test << endl;
21
22     return 0;
23 }
```

32767
-32768
32767

inf
0

Định dạng nhập xuất

- ❑ Căn độ rộng
- ❑ Đặt độ chính xác

Ví dụ 2a: Nhu cầu căn độ rộng

```
1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      int num1 = 2897, num2 = 5, num3 = 837, num4 = 34, num5 = 7,
7      |   num6 = 1623, num7 = 390, num8 = 3456, num9 = 12;
8      // Display the first row of numbers
9      cout << num1 << " ";
10     cout << num2 << " ";
11     cout << num3 << endl;
12     // Display the second row of numbers
13     cout << num4 << " ";
14     cout << num5 << " ";
15     cout << num6 << endl;
16     // Display the third row of numbers
17     cout << num7 << " ";
18     cout << num8 << " ";
19     cout << num9 << endl;
20     return 0;
21 }
```

Program Output

2897 5 837

34 7 1623

390 3456 12

Căn độ rộng với setw()

- Đa số hàm định dạng nằm trong thư viện <iomanip>
- Hàm setw(int n)
 - Dùng nối với cout << , ví dụ cout << setw(5)
 - Chỉ áp lên đối tượng **ngay kế tiếp**, hết tác dụng với đối tượng sau đó
 - Dành chỗ n ký tự trên màn hình cho đối tượng, **mặc định căn theo lề phải** (có thể đổi nếu muốn).
 - Có thể gọi nhiều lần trong cùng một lệnh cout

Ví dụ 2b: Căn độ rộng với setw()

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main()
6  {
7      int num1 = 2897, num2 = 5, num3 = 837, num4 = 34, num5 = 7,
8          num6 = 1623, num7 = 390, num8 = 3456, num9 = 12;
9
10     // Display the first row of numbers
11     cout << setw(8) << num1 << " "
12         << setw(8) << num2 << " "
13         << setw(8) << num3 << endl;
14     // Display the second row of numbers
15     cout << setw(8) << num4 << " "
16         << setw(8) << num5 << " "
17         << setw(8) << num6 << endl;
18     // Display the third row of numbers
19     cout << setw(5) << setw(4) << setw(8) << num7 << " "
20         << setw(8) << num8 << " "
21         << setw(8) << num9 << endl;
22
23     return 0;
24 }
```

Program Output

2897	5	837
34	7	1623
390	3456	12

Gọi liên tiếp setw()

Lấy hiệu ứng của
setw() cuối cùng

Đặt độ chính xác với `setprecision()`, `showpoint`, `fixed`

- Độ chính xác của số thực được thể hiện bằng nhiều cách: số chữ số sau phẩy, số chữ số biểu diễn (cả trước và sau phẩy), hay số chữ số có nghĩa (số chữ số trước phẩy cộng số chữ số sau phẩy tính đến trước khi toàn 0).
- Thiết lập độ chính xác bằng các hàm <iomanip>:
 - ❑ `cout << setprecision(n)`: hiện nhiều nhất n chữ số **có nghĩa**
 - ❑ `cout << setprecision(n) << showpoint`: hiện n chữ số **tổng trước và sau phẩy**, luôn hiện dấu phẩy
 - ❑ `cout << setprecision(n) << fixed`: hiện n chữ số **sau phẩy**
- Áp dụng cho **mọi số thực in ra** cho đến khi có thiết lập mới.

Setprecision vs. showpoint vs. fixed

Giá trị	setprecision(5)	setprecision(5) thêm cờ showpoint	Setprecision (5) thêm cờ fixed
	<i>Cả trước và sau phẩy Chỉ tính chữ số có nghĩa Nhiều nhất n chữ số</i>	<i>Cả trước và sau phẩy Tính mọi chữ số Đúng n chữ số Luôn hiện dấu chấm</i>	<i>Chỉ sau phẩy Tính mọi chữ số Đúng n chữ số</i>
300	300	300.00	300.00000
300.01	300	300.01	300.01000
300.10	300.1	300.10	300.10000
300.123	300.12	300.12	300.12300
300.123456	300.12	300.12	300.12346
30000.1	30000	30000.	30000.10000
300000.1	3e+05	3.0000e+05	300000.10000

Ví dụ 2c: setprecision(), showpoint, fixed

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main()
6  {
7      const long double quotient = 3.12301;
8      cout << quotient << endl;
9      cout << "Use setprecision:" << endl;
10     cout << setprecision(5) << quotient << endl;
11     cout << setprecision(4) << quotient << endl;
12     cout << setprecision(3) << quotient << endl;
13     cout << setprecision(2) << quotient << endl;
14     cout << setprecision(1) << quotient << endl;
15     // cout << setprecision(0) << quotient << endl;
16     // cout << setprecision(-1) << quotient << endl;
17     cout << "Use setprecision(5) with showpoint:" << endl;
18     cout << setprecision(5) << showpoint << quotient << endl;
19     cout << "Use setprecision(5) with fixed:" << endl;
20     cout << fixed << quotient << endl;
21     return 0;
22 }
```

3.12301
Use setprecision:
3.123
3.123
3.12
3.1
3
Use setprecision(5) with showpoint:
3.12301
Use setprecision(5) with fixed:
3.12301

Vì sao cuối cùng ra
kết quả giống nhau?

Hãy đoán kết quả!!

Không setprecision(5)
nữa vì dùng tiếp

Đọc thêm: Cờ vào ra, hàm thành viên

- Thay vì gọi `showpoint` và `fixed`, có thể định dạng dữ liệu xuất bằng cách thiết lập cờ vào ra:

`cout << setiosflag(<tên cờ>)`

- Đặt cờ `ios::fixed` tương đương `fixed`
- Đặt cờ `ios::showpoint` tương đương `showpoint`
- Đặt hỗn hợp `ios::fixed | ios::showpoint`: `setprecision(n)` sẽ tính số chữ số sau phẩy + luôn hiện phẩy dù giá trị có nguyên
- Một cách nữa là dùng các hàm thành viên của đối tượng `cout`:

```
cout.width(5);    //same as setw(5)
cout.precision(2); //same as setprecision(2)
cout.setf(ios::fixed); //same as setiosflags(ios::fixed)
```

Ví dụ 2d: Thiết lập cờ vào ra

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4  int main()
5  {
6      float day1, day2, day3, total;
7      cout << "Enter the sales for day 1: ";
8      cin >> day1;
9      cout << "Enter the sales for day 2: ";
10     cin >> day2;
11     cout << "Enter the sales for day 3: ";
12     cin >> day3;
13     total = day1 + day2 + day3;
14     cout << "\nSales Figures\n";
15     cout << "-----\n";
16     cout << setprecision(2) << setiosflags(ios::fixed | ios::showpoint);
17     cout << "Day 1: " << setw(8) << day1 << endl;
18     cout << "Day 2: " << setw(8) << day2 << endl;
19     cout << "Day 3: " << setw(8) << day3 << endl;
20     cout << "Total: " << setw(8) << total << endl;
21
22     return 0;
23 }
```

Program Output

```
Enter the sales for day 1: 2642.00 [Enter]
Enter the sales for day 2: 1837.20 [Enter]
Enter the sales for day 3: 1963.00 [Enter]
```

Sales Figures

```
-----
Day 1:    2642.00
Day 2:    1837.20
Day 3:    1963.00
Total:    6442.20
```

Setw cần lặp lại cho từng đối tượng in ra, còn setprecision và setiosflag chỉ cần 1 lần, có tác dụng cho đến khi được đặt lại

Các cờ vào ra

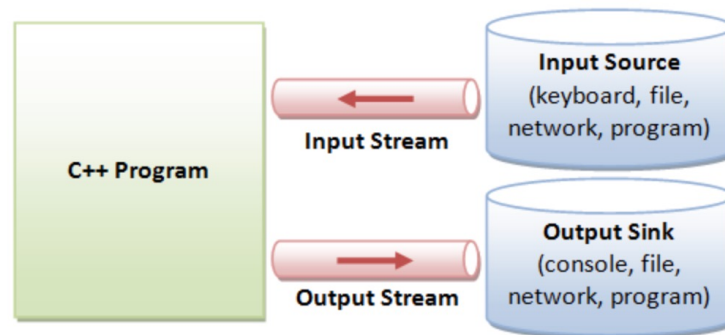
Format Flag	Description
<code>ios::left</code>	Causes the field to be left-justified with padding spaces printed to the right.
<code>ios::right</code>	Causes the field to be right-justified with padding spaces printed to the left.
<code>ios::fixed</code>	Causes all subsequent numbers to be displayed in fixed point notation.
<code>ios::scientific</code>	Causes all subsequent numbers to be displayed in scientific notation.
<code>ios::dec</code>	Causes all subsequent integers to be displayed in decimal format.
<code>ios::hex</code>	Causes all subsequent integers to be displayed in hexadecimal format.
<code>ios::oct</code>	Causes all subsequent integers to be displayed in octal format.
<code>ios::showpoint</code>	Forces the decimal point and trailing zeroes to be displayed.
<code>ios::showpos</code>	Causes a + sign to be displayed in front of positive numbers.
<code>ios::uppercase</code>	Causes the E in scientific notation numbers, and the X in hexadecimal numbers to be displayed in uppercase.

Làm việc với xâu và ký tự

- ❑ Dòng dữ liệu
- ❑ Đọc xâu và ký tự từ dòng
- ❑ Gỡ lỗi khi đọc dòng

Dòng dữ liệu

- Dữ liệu giống như nước, nước có thể từ đâu đó chảy vào chương trình (nhập liệu) hay từ chương trình chảy ra đâu đó (xuất liệu).
- Bàn phím là nguồn nước, nguồn đổ vào các ống (cin, getline, ...) thành dòng chảy vào chương trình, cuối dòng chính là ký hiệu \n
- Ta vớt thông tin - phần dữ liệu ý nghĩa từ dòng nước, lưu vào biến
- Ta không thể kiểm soát nguồn hay dòng, chỉ có thể kiểm soát ống
- Làm thế nào khi ống tắc, khi nước bắn, hay khi ống đầy?
- Chương trình có thể quyết định mở, đóng, hay xả nước khỏi ống



Đọc xâu

- Xâu: là một chuỗi liên tục ký tự
 - khai báo như mảng char hoặc như string `char myCharString[80];`
`string myString;`
- Có nhiều lệnh đọc xâu với các tính năng khác nhau:
 - Điều vớt dữ liệu ống lưu vào xâu => phải đợi trong ống có dữ liệu
 - `cin >> s` : nhảy qua dấu trắng để vớt xâu không rỗng không trắng đầu tiên, dừng ở dấu trắng kế, để lại mọi thứ từ đó trong ống

Ví dụ 3a: cin với string

```
// This program uses cin with string
#include <iostream>
using namespace std;
int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;
    cout << "String: " << s << endl;
    cout << s[3] << endl;

    return 0;
}
```

Enter a string:

abcde fgh ikj
String: abcde
d

Một string gần giống một mảng ký tự, nên có thể truy cập từng ký tự như phần tử mảng.

Ví dụ 3b: cin với mảng char

```
// This program uses cin with char []
#include <iostream>
using namespace std;
int main()
{
    char array[5];
    cout << "Enter a char array:";
    cin >> array;
    cout << "Array:" << array << endl;
    cout << array[2] << endl ;

    return 0;
}
```

Sẽ ra sao nếu nhập quá số ký tự của mảng?
Sẽ ra sao nếu truy cập quá chỉ số mảng? .

Enter a char array:
abcd efgh
Array:abcd
c

Ví dụ 3b: cin với mảng char

```
// This program uses cin with char []
#include <iostream>
using namespace std;
int main()
{
    char array[5];
    cout << "Enter a char array:";
    cin >> array;
    cout << "Array:" << array << endl;
    cout << array[5] << endl ;

    return 0;
}
```

Enter a char array:abcdefgh
Array:abcdefgh
f

warning: array index 5 is past the end of the array

Đọc sâu

- Xâu: là một chuỗi liên tục ký tự
 - khai báo như mảng char hoặc như string
- Có nhiều lệnh đọc xâu với các tính năng khác nhau:
 - Đọc dữ liệu ống lưu vào xâu => phải đợi trong ống có dữ liệu
 - `cin >> s` : nhảy qua dấu trắng để vớt xâu không rỗng không trắng đầu tiên, dừng ở dấu trắng kế, để lại mọi thứ từ đó trong ống
 - `cin >> setw(n)>>s` : giống `cin`, nhưng bên cạnh dừng nếu gặp dấu trắng kế, còn dừng khi đã vớt đủ ký tự ($n-1$ với mảng, n với string)

```
char myCharString[80];  
string myString;
```

Ví dụ 3c: cin>>setw(n) với mảng char

```
// This program uses cin >> setw() with char []
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    char array[5];
    cout << "\nEnter a char array: ";
    cin >> setw(5) >> array;
    cout << "Array:" << array << endl;
    cout << "array[3]: " << array[3] << endl;
    return 0;
}
```

Kết quả sẽ ra sao nếu nhập:
Sha la la [Enter]
Sẽ ra sao nếu truy cập quá chỉ số
mảng?

Enter a char array: eureka
Array:eure
array[3]: e

Enter a char array: Sha la la
Array:Sha
array[3]:

Ví dụ 3d: cin>>setw(n) với string

```
// This program uses cin >> setw() with string
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> setw(5) >> s;
    cout << "String: " << s << endl;
    cout << "s[3]: " << s[3] << endl;
    cout << "s[4]: " << s[4] << endl;
    cout << "s[7]: " << s[7] << endl;
    return 0;
}
```

```
Enter a string: abcdefg
String: abcde
s[3]: d
s[4]: e
s[7]:
```

warning: array index 5 is past the end of the array

Đọc sâu

- Xâu: là một chuỗi liên tục ký tự
 - khai báo như mảng char hoặc như string
- Có nhiều lệnh đọc xâu với các tính năng khác nhau:
 - `cin >> s`: nhảy qua dấu trắng để vớt xâu không rỗng không trắng đầu tiên, dừng ở dấu trắng kế, để lại mọi thứ từ đó trong ống
 - `cin >> setw(n)>>s`: giống `cin`, nhưng bên cạnh dừng nếu gặp dấu trắng kế, còn dừng khi đã vớt đủ ký tự ($n-1$ với mảng, n với string)
 - `getline (cin, s, delim)`: vớt từ đầu không nhảy qua dấu trắng, dừng vớt khi gặp ký tự ngắt `delim`, nên xâu kết quả được quyền rỗng, được quyền chứa dấu trắng; ngoài ra tuy không lấy `delim` vẫn xóa `delim` khỏi ống, chỉ để lại ống mọi thứ sau `delim`.

```
char myCharString[80];  
string myString;
```

Ví dụ 3e: getline(cin, s, delim)

```
1  // This program uses getline(cin, s, delim)
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      string sentence;
7
8      cout << "Enter a sentence: ";
9      getline(cin, sentence); //delim = '\n'
10     cout << "You entered " << sentence << endl;
11
12     return 0;
13 }
```

Tham số mặc định: Khi không truyền đối số cho tham số, giá trị mặc định của tham số sẽ được sử dụng, ở đây hàm getline có tham số mặc định là '\n' (tra cứu tutorial)

Enter a sentence: **To be, or not to be, that is the question. [Enter]**
You entered To be, or not to be, that is the question.

Ví dụ 3e: getline(cin, s, delim)

```
1  // This program uses getline(cin, s, delim)
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      string sentence;
7
8      cout << "Enter a sentence: ";
9      getline(cin, sentence); //delim = '\n'
10     cout << "You entered " << sentence << endl;
11
12     return 0;
13 }
```

Enter a sentence: **[Enter]**
You entered

Ví dụ 3f: getline(cin, s, delim)

```
1  // This program uses getline(cin, s, delim)
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      string sentence;
7
8      cout << "Enter a sentence: ";
9      getline(cin, sentence, ','); //delim = ','
10     cout << "You entered " << sentence << endl;
11
12     return 0;
13 }
```

Enter a sentence: **To be, or not to be, that is the question. [Enter]**
You entered To be

Đọc sâu, đọc ký tự

- Xâu: là một chuỗi liên tục ký tự
 - khai báo như mảng char hoặc như string
- Có nhiều lệnh đọc xâu với các tính năng khác nhau:
 - Điều vớt dữ liệu ống lưu vào xâu => phải đợi trong ống có dữ liệu
 - `cin >> s` : nhảy qua dấu trắng để vớt xâu không rỗng không trắng đầu tiên, dừng ở dấu trắng kế, để lại mọi thứ từ đó trong ống
 - `cin >> setw(n)>>s` : giống `cin`, nhưng bên cạnh dừng nếu gặp dấu trắng kế, còn dừng khi đã vớt đủ ký tự ($n-1$ với mảng, n với string)
 - `getline (cin, s, delim)`: vớt từ đầu không nhảy qua dấu trắng, dừng vớt khi gặp ký tự ngắt `delim`, nên xâu kết quả được quyền rỗng, được quyền chứa dấu trắng; ngoài ra tuy không lấy `delim` vẫn xóa `delim` khỏi ống, chỉ để lại ống mọi thứ sau `delim`.
- `cin.get (c)` vớt 1 ký tự đầu tiên, kể cả dấu trắng, để lại mọi thứ từ đó trở đi. VD: gõ enter luôn thì chương trình thu được `c = '\n'`

```
char myCharString[80];  
string myString;
```

Ví dụ 3g: cin.get(c)

```
// This program uses cin.get(c)
#include <iostream>
using namespace std;
int main()
{
    char c;
    cout << "\nEnter a char: ";
    cin.get(c);
    cout << "Your char is " << c << "." << endl << endl;

    return 0;
}
```

Enter a char: a
Your char is a.

Enter a char: abc
Your char is a.

Enter a char:
Your char is .

Enter a char:
Your char is
.

Các vấn đề khi đọc dòng – Gỡ lỗi

- Kết dòng nhập thường xuyên là dấu enter.
- Nếu một lệnh nhập để dấu này trong dòng không dọn đi, mà lệnh nhập ngay sau lại không biết nhảy qua enter, thì dấu enter bị bỏ lại này có thể làm lệnh sau kết thúc luôn và nhảy đến lệnh kế, dù người dùng chưa nhập gì thêm.
- VD: Một vài tình huống kiểu trên?

Ví dụ 4a: cin rồi getline(cin, s)

```
1  // This program uses getline(cin, s, delim)
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      char c;
7      cout << endl << "Enter a char: ";
8      cin >> c;
9      cout << "char is : " << c << endl ;
10
11     string sentence;
12     cout << endl << "Enter a sentence: ";
13     getline(cin, sentence); //delim = '\n'
14     cout << "You entered " << sentence << endl << endl;
15
16     // char c;
17     // cout << "Enter a char: ";
18     // cin >> c;
19     // cout << "char is : " << c << endl << endl;
20     return 0;
21 }
```

Enter a char: ab
char is : a
Enter a sentence: You entered b

Enter a char: a
char is : a
Enter a sentence: You entered

Ví dụ 4a: getline(cin, s) rồi cin

```
1  // This program uses getline(cin, s, delim)
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      // char c;
7      // cout << endl << "Enter a char: ";
8      // cin >> c;
9      // cout << "char is : " << c << endl ;
10
11     string sentence;
12     cout << endl << "Enter a sentence: ";
13     getline(cin, sentence); //delim = '\n'
14     cout << "You entered " << sentence << endl << endl;
15
16     char c;
17     cout << "Enter a char: ";
18     cin >> c;
19     cout << "char is : " << c << endl << endl;
20     return 0;
21 }
```

Enter a sentence: ab cd e
You entered ab cd e

Enter a char: a
char is : a

Ví dụ 4b: cin rồi cin.get(c)

```
// This program uses cin.get(c)
#include <iostream>
using namespace std;
int main()
{
    char c1, c2;
    cout << "Enter a char: ";
    cin >> c1;

    cout << "\nEnter a char: ";
    cin.get(c2);
    cout << "\nYour char is " << c1 << " and " << c2 << endl ;

    return 0;
}
```

Enter a char: e

Enter a char:
Your char is e and

Các vấn đề khi đọc dòng – Gỡ lỗi

- Kết dòng nhập luôn là dấu enter. Nếu một lệnh nhập để dấu này trong dòng không dọn đi, mà lệnh nhập ngay sau lại không biết nhảy qua enter, thì dấu enter bị bỏ lại này có thể làm lệnh sau kết thúc luôn và nhảy đến lệnh kế, dù người dùng chưa nhập gì thêm
- VD: Các lệnh `cin`, `cin>>setw`, `cin.get()` để nguyên mọi thứ nó không dọn trong dòng, bao gồm cả dấu enter. Nếu dùng lệnh `getline` sau đó, lệnh này vốn không biết nhảy qua dấu trắng, thì sẽ có vấn đề.
- Ngược lại, `getline(cin, s)` đọc xong tự dọn ký tự ngắt enter khỏi dòng nhập nên các lệnh sau không sợ gặp phải enter thừa. Nên ngay sau `getline` dùng bất kỳ lệnh gì đều không sao cả.
- Cách giải quyết: ?

Ví dụ 4c: cin.ignore(1000, '\n')

```
// This program uses getline(cin, s, delim) and cin.ignore (1000, '\n')
#include <iostream>
using namespace std;
int main()
{
    char c;
    cout << endl << "Enter a char: ";
    cin >> c;
    cout << "char is : " << c << endl ;

    cin.ignore(1000, '\n');

    string sentence;
    cout << endl << "Enter a sentence: ";
    getline(cin, sentence); //delim = '\n'
    cout << "You entered " << sentence << endl << endl;

    return 0;
}
```

Enter a char: ab
char is : a

Enter a sentence: Now it's fine!
You entered Now it's fine!

Enter a char: a
char is : a

Enter a sentence: Now it's also fine.
You entered Now it's also fine.

Ví dụ 4d: cin.ignore()

```
1  // This program uses cin.get(c) and cin.ignore()
2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6      cout << "\nEnter a char: ";
7      char c1, c2;
8      cin >> c1;
9      cin.ignore();
10     cout << "\nEnter a char: ";
11     cin.get(c2);
12     cout << "\nYour char is " << c1 << " and " << c2 << endl;
13
14     return 0;
15 }
```

Enter a char: e

Enter a char: d

Your char is e and d

Các vấn đề khi đọc dòng – Gỡ lỗi

- Kết dòng nhập luôn là dấu enter. Nếu một lệnh nhập để dấu này trong dòng không dọn đi, mà lệnh nhập ngay sau lại không biết nhảy qua enter, thì dấu enter bị bỏ lại này có thể làm lệnh sau kết thúc luôn và nhảy đến lệnh kế, dù người dùng chưa nhập gì thêm
- VD: Các lệnh `cin`, `cin>>setw`, `cin.get()` để nguyên mọi thứ nó không dọn trong dòng, bao gồm cả dấu enter. Nếu dùng lệnh `getline` sau đó, lệnh này vốn không biết nhảy qua dấu trắng, thì sẽ có vấn đề.
- Ngược lại, `getline(cin, s)` đọc xong tự dọn enter khỏi dòng nhập nên các lệnh sau không gặp phải enter thừa, sau `getline` dùng bất kỳ lệnh gì đều không sao cả.
- Tóm lại, dữ liệu chưa vớt hết mà chưa rõ để làm gì thì giống như nước tù đọng, có thể gây vấn đề, tốt nhất nên chủ động xả đi:
 - Dùng `cin.ignore()`: nhảy qua một ký tự kế trong dòng nhập; hoặc
 - `cin.ignore (1000, '\n')`: nhảy qua 10000 ký tự hoặc đến khi gặp '\n'
- **Tự tìm hiểu:** Có lúc nhập sai kiểu, vượt cỡ xâu,.. làm ống `cin` bật cờ lỗi và khoá dòng không thể đọc tiếp. Để hạ cờ lỗi, dùng `cin.clear()`

Đọc thêm: Một số hàm hữu ích

- ❑ Thư viện `<cstdlib>`
- ❑ Thư viện `<math>`

Thư viện <cstdlib>

- Hàm dừng màn hình system (“pause”)
 - sẽ dừng màn hình khi chạy, ấn Enter để tiếp tục,
 - chỉ dùng cho máy Window
- Hàm sinh số ngẫu nhiên rand()
 - Trước đó gọi srand(n) với n là số nguyên không dấu, để nhập n làm hạt giống sinh ngẫu nhiên
 - Cách tốt là lấy chính giờ hệ thống làm hạt giống, cần thêm thư viện <ctime> và dùng srand(time(0))

Thư viện `<math>`

- `abs(x)`: giá trị tuyệt đối của x
- `sqrt(x)`: căn bậc 2 của x
- `pow(b, x)`: cơ số b mũ x
- `exp(x)`: cơ số e mũ x
- `log(x)`: loga cơ số e của x
- `log10(x)`: loga cơ số 10 của x
- `sin(x)`, `cos(x)`, `tan(x)`: $\sin x$, $\cos x$, $\tan x$

Tổng kết

- Sự định kiểu của biến, hằng, literal, biểu thức
- Vi phạm kiểu do không tương thích: ép kiểu ngầm định và ép kiểu tường minh
- Vi phạm kiểu do sai phạm vi: tràn và hụt
- Cách định dạng: đặt độ rộng, số chữ số sau phẩy
- Làm việc với chuỗi và ký tự, các lỗi và cách gỡ lỗi
- Các hàm hữu ích của thư viện math và cstdlib

Trò chơi đoán số (tiếp)

Máy tính nghĩ ra một số ngẫu nhiên trong khoảng từ 1 đến 1000, nó giữ bí mật và mời bạn đoán. Mỗi lần đoán, nó sẽ cho bạn thắng nếu bạn trả lời đúng số cần đoán. Nếu không đúng, bạn được biết giá trị vừa đoán là quá cao hay quá thấp so với số cần đoán. Sau 10 đoán mà vẫn không đoán trúng thì bạn sẽ thua.

Hello! What is your name?

Mai

Play with me, Mai! I am thinking of a number between 1 and 1000.

Take a guess.

10

Your guess is too high.

Take a guess.

2

Your guess is too low.

Take a guess.

4

Well done, Mai! You guessed my number in 3 guesses!

Trò chơi đoán số - Tuần 2

- Tuần này ta có thể mở rộng lời chào tuần trước để nhập đầy đủ tên lẫn họ (tức là một xâu có chứa dấu cách)
- Ta có thể bắt đầu nói máy tính sinh thử ra một số ngẫu nhiên trong khoảng từ 1 đến n (gợi ý: nhờ dùng hàm rand())
- Ngoài ra trong tương lai ta có lẽ không chỉ muốn chơi đoán số từ 1 đến 1000, hay số lần đoán không nhất thiết là 10.
- Bonus: Cuối cùng, ta sẽ cần tính tới cả trường hợp người dùng vô tình nhập xâu thay vì số
- Hãy cập nhật chương trình GuessNumber đã làm tuần 1 để bổ sung những mở rộng trên

Trò chơi đoán số - Tuần 2 - (chữa)

```
#include <iostream>
#include <ctime>    // hàm time
#include <cstdlib>  // hàm srand, rand
using namespace std;
int main(){
    string name;
    int n, guess;
    srand(time(0));    // sinh hạt giống để sinh số ngẫu nhiên
    n = rand()%(1000) + 1; // sinh số ngẫu nhiên trong [1, 1000]

    cout << "Xin chào. Tên bạn là gì? \n";
    getline(cin, name);    // đọc tên có dấu cách
    cout << "Chào bạn " << name << ". Tôi đang nghĩ về một số trong khoảng từ " << 1 << " đến " << 1000 << ".\n"
    << "Bạn hãy thử đoán! \n";

    if (cin >> guess) cout << "Bạn vừa đoán số " << guess << ".\n"; // cin >> guess sẽ true nếu nhập đúng
    else{
        cin.clear();    // hạ cờ lỗi
        cin.ignore(100, '\n'); // nhảy qua dữ liệu thừa nếu có
        cout << "Bạn đã nhập sai 1 lần, mọi bạn nhập lại: \n";
        if (cin >> guess) cout << "Bạn vừa đoán số " << guess << ". \n";
        else cout << "Bạn đã nhập sai 2 lần. Trò chơi kết thúc\n";
    }
    return 0;
}
```

Trò chơi đoán số - Tuần 2 - (chữa)

- Để nhập tên đầy đủ dùng `getline(cin, s)`
- Dùng trộn `cin` và `getline (cin,...)` thì nên nhớ xả dòng nhập
 - Trường hợp đơn giản: chỉ cần `cin.ignore(100, '\n')`
 - Trường hợp yêu cầu ngặt nghèo hơn:
`cin.ignore (numeric_limits <streamsize> :: max(), '\n')`
 - Nhảy qua cỡ cực đại của bộ đệm nhập, không sợ nhảy thiếu
 - Có thể cho vào hàm riêng để tiện gọi nhiều lần
- Để các tham số dễ thay đổi về sau, cho chúng thành biến hay hằng

Trò chơi đoán số - Tuần 2 - (chữa)

- Xử lý dữ liệu lỗi = lỗi nào? + cách giải quyết nào?
- Các loại lỗi
 - Vót được thông tin mà vô nghĩa, vd cần số ≥ 0 mà nhập âm)
 - Bị thừa thông tin, vd cần một ký tự mà nhập một chuỗi
 - Không vót được thông tin do không tương thích kiểu
 - Không vót được thông tin do vượt quá phạm vi
- Các cách giải quyết:
 - Chặn người dùng nhập lỗi
 - Lệnh cơ bản của C++ không cung cấp sẵn
 - Để nhập lỗi, sau đó trích dữ liệu đúng từ dữ liệu lỗi
 - Phức tạp, quá nhiều trường hợp, có lúc không thể trích
 - Để nhập lỗi, phát hiện lỗi, khắc phục hậu quả, mời nhập lại
 - Đơn giản, đủ tốt

Trò chơi đoán số - Tuần 2 - (chữa)

```
#include <iostream>
#include <ctime>    // hàm time
#include <cstdlib>  // hàm srand, rand
using namespace std;
int main(){
    const int MIN = 1, MAX= 1000, N = 10;
    string name;
    int n, guess;
    srand(time(0));           // sinh hạt giống để sinh số ngẫu nhiên
    n = rand()%(MAX + 1 - MIN) + MIN; // sinh số ngẫu nhiên trong [MIN, MAX]

    cout << "Xin chào. Tên bạn là gì? \n";
    getline(cin, name);       // đọc tên có dấu cách
    cout << "Chào bạn " << name << ". Tôi đang nghĩ về một số trong khoảng từ " << MIN << " đến " << MAX << ".\n"
    << "Bạn hãy thử đoán! \n";

    if (cin >> guess) cout << "Bạn vừa đoán số " << guess << ".\n"; // cin >> guess sẽ true nếu nhập đúng
    else{
        cin.clear();           // hạ cờ lỗi
        cin.ignore(100, '\n'); // nhảy qua dữ liệu thừa nếu có
        cout << "Bạn đã nhập sai 1 lần, mời bạn nhập lại: \n";
        if (cin >> guess) cout << "Bạn vừa đoán số " << guess << ". \n";
        else cout << "Bạn đã nhập sai 2 lần. Trò chơi kết thúc\n";
    }
    return 0;
}
```